

Introduction to Algorithms

Personal Notes

Mihir Rao

Contents

1	Introduction	2
1.1	Problems and Algorithms	2
1.2	A Running Example: Birthday Matching	2
1.3	Correctness	3
1.4	Efficiency and Asymptotic Growth	3
1.5	The Word-RAM Model	4
1.6	Data Structures and Interfaces	5
1.7	A Strategy for Algorithmic Problems	5

1 Introduction

Algorithms turn computational questions into procedures whose answers can be checked and whose resource use can be understood. The goal of this course is therefore twofold: learn to solve broad families of computational problems, and learn to communicate why a proposed solution is both correct and efficient.

1.1 Problems and Algorithms

Definition 1.1 (Computational problem). A *computational problem* is a binary relation between a set of inputs and a set of acceptable outputs. Equivalently, each input has one or more outputs that satisfy a verifiable correctness condition.

A problem describes an entire family of instances, not one particular case. For example, asking whether two people in a specific classroom share a birthday is a single instance. Asking whether any collection of n people contains a matching pair is a general problem whose input size can grow.

Note: The input-output relation need not select a unique answer. A search problem may allow several correct outputs; the specification only needs to make every candidate answer verifiable.

Definition 1.2 (Algorithm). An *algorithm* is a deterministic procedure that maps every valid input to one output. It *solves* a problem when that output satisfies the problem's correctness condition for every valid input.

The universal quantifier is essential. Producing the right answer on examples is evidence, but an algorithm is a solution only if it works on inputs of every allowed size and shape.

1.2 A Running Example: Birthday Matching

Suppose the input is a sequence of students, where each student has a name and a birthday. The task is to return two names with the same birthday, or report that no such pair exists. A direct algorithm keeps the students already seen in a record and compares each new birthday against that record.

```
1 def birthday_match(students):
2     record = [None] * len(students)
3     for k, (name, birthday) in enumerate(students):
4         for i in range(k):
5             if birthday == record[i][1]:
6                 return (name, record[i][0])
7         record[k] = (name, birthday)
8     return None
```

This program is deliberately simple. Later data structures can make the search faster, but the nested-loop version cleanly separates the two obligations that every algorithm carries: correctness and efficiency.

1.3 Correctness

Testing cannot cover an arbitrarily large input space. Because a finite program typically handles large inputs by repeating a loop or invoking itself recursively, induction is the natural way to reason about its behavior.

Proposition 1.3. *The birthday-matching algorithm returns a matching pair if one exists, and returns `None` otherwise.*

Proof. Induct on k , the number of students already stored in record. Use the invariant that, before student k is processed, the algorithm would already have returned if the first k students contained a matching pair.

For $k = 0$, no pair exists, so the invariant holds. Assume it holds before processing student k . If a match occurred entirely among the first k students, the induction hypothesis says the algorithm has already returned it. Otherwise, any match among the first $k + 1$ students must include the new student. The inner loop compares that student's birthday with every earlier birthday, so it returns exactly such a pair if one exists. If no comparison matches, storing the new student preserves the invariant.

After all n students have been processed, the invariant rules out any match if the algorithm reaches its final line. Thus returning `None` is also correct. $\square$

1.4 Efficiency and Asymptotic Growth

Wall-clock time depends on the processor, programming language, compiler, and other implementation details. Algorithm analysis instead counts elementary operations and expresses that count as a function of input size, usually n . This gives a machine-independent description of how resource use scales.

Definition 1.4 (Asymptotic bounds). For nonnegative functions f and g :

- $f(n) = O(g(n))$ gives an asymptotic upper bound;
- $f(n) = \Omega(g(n))$ gives an asymptotic lower bound; and
- $f(n) = \Theta(g(n))$ gives matching upper and lower bounds.

These bounds ignore constant factors and lower-order terms.

Common growth rates form a hierarchy:

Growth	Typical name
$\Theta(1)$	constant
$\Theta(\log n)$	logarithmic
$\Theta(n)$	linear
$\Theta(n \log n)$	linearithmic
$\Theta(n^2)$	quadratic
$\Theta(n^c)$	polynomial, for constant c
$2^{\Theta(n^c)}$	exponential

The difference is qualitative, not cosmetic. If $n = 1000$ and a machine performs roughly one billion elementary operations per second, a linear algorithm takes about a microsecond, an $n \log_2 n$ algorithm about ten microseconds, and a quadratic algorithm about a millisecond. An algorithm with 2^n growth is already far beyond reach.

For the birthday algorithm, the outer loop runs n times. On iteration k , the inner loop performs at most k comparisons. Therefore

$$T(n) = \Theta(n) + \sum_{k=0}^{n-1} \Theta(k) = \Theta(n) + \Theta\left(\frac{n(n-1)}{2}\right) = \Theta(n^2).$$

This is polynomial time, but the course will show how choosing a better data structure for the record improves the search substantially.

1.5 The Word-RAM Model

Counting operations only makes sense after fixing which operations count as constant time. A *model of computation* provides that contract. The course uses the Word-RAM model.

Definition 1.5 (Word-RAM). A w -bit *Word-RAM* consists of an addressable memory containing w -bit words and a processor that can operate on a constant number of words in constant time.

The allowed operations include integer arithmetic, comparisons and Boolean operations, bit operations, and reading or writing the memory word at a given address. A memory address itself must fit in one word. If the input occupies $\Theta(n)$ words, this requires

$$w \geq \lceil \log_2 n \rceil.$$

Thus the model permits constant-time arithmetic on values large enough to index the input, but it does not declare arbitrarily large integer operations free.

***Note:** Python runs on real machines ultimately modeled by word-level operations, but a single Python operation can hide nonconstant work. The Word-RAM abstraction is useful because it makes such cost assumptions explicit.*

1.6 Data Structures and Interfaces

Definition 1.6 (Data structure). A *data structure* stores a nonconstant amount of data while supporting a specified collection of operations. That collection of operations is the data structure's *interface*.

An interface says what clients may do; an implementation determines the cost. A sequence interface, for example, exposes positions such as first, last, or *i*th. A set interface instead organizes access around intrinsic keys. Several data structures may implement the same interface with different performance tradeoffs.

A static array is the first example. It contains a fixed number of fixed-width slots:

- allocating and initializing n slots takes $\Theta(n)$ time;
- reading the word at a known index takes $\Theta(1)$ time; and
- writing the word at a known index takes $\Theta(1)$ time.

A word may also hold an address to a larger object. Conceptually, a static array resembles a fixed-length Python tuple with an additional constant-time write operation; Python's list is instead a dynamic array.

1.7 A Strategy for Algorithmic Problems

When approaching a new problem, first try to reduce it to one already solved. A data structure may provide the needed search operation, a sorting algorithm may expose useful order, or a graph algorithm may capture the relevant relationships. If no reduction fits, design around a familiar paradigm such as brute force, decrease-and-conquer, divide-and-conquer, dynamic programming, or an incremental greedy method.

Source. These notes synthesize Lecture 1 of MIT OpenCourseWare's 6.006 Introduction to Algorithms, Spring 2020, taught by Erik Demaine, Jason Ku, and Justin Solomon.